

Flask - WTF

Flask-WTF is a Flask extension that integrates the WTForms library, making form creation and validation easier in Flask applications. It provides a structured way to build forms, handle validation, and render them in HTML.

Some of the key features of Flask-WTF include:

- Secure Form Handling – Automatically manages CSRF protection to prevent unauthorized submissions.
- Easy Form Rendering – Supports various field types like text fields, checkboxes, and dropdowns for smooth HTML integration.
- Built-in Validation – Includes required fields, length constraints, pattern matching, and support for custom validation.
- File Uploads – Allows users to upload files through forms seamlessly.

Installation

To use Flask-WTF, we first need to install it using pip and import it into our application.

```
pip install flask-WTF
```

In Flask-WTF, forms are defined as classes that extend the FlaskForm class. Fields are declared as class variables, making form creation simple and structured.

Common WTForms Field Types:

- StringField: Text input field for string data.
- PasswordField: Input field for password values.
- BooleanField: Checkbox for True/False selection.
- DecimalField: Input field for decimal values.
- RadioField: Group of radio buttons for single selection.
- SelectField: Dropdown list for single selection.
- TextAreaField: Multi-line text input field.
- FileField: File upload field.

For example,

form.py

```
from flask import Flask, render_template, request
```

```

from flask_wtf import FlaskForm
from wtforms import StringField, PasswordField, BooleanField
from wtforms import DecimalField, RadioField, SelectField, TextAreaField,
FileField
from wtforms.validators import InputRequired, Length
from werkzeug.security import generate_password_hash

app = Flask(__name__)
app.config['SECRET_KEY'] = 'secretkey'

class MyForm(FlaskForm):
    name = StringField('Name', validators=[InputRequired(), Length(min=3,
max=15)])
    password = PasswordField('Password', validators=[InputRequired()])
    remember_me = BooleanField('Remember me')
    salary = DecimalField('Salary', validators=[InputRequired()])
    gender = RadioField('Gender', choices=[
        ('male', 'Male'), ('female', 'Female')])
    country = SelectField('Country', choices=[('N', 'Nepal'), ('I', 'India'),
        ('C', 'China'), ('B', 'Bhutan')])
    message = TextAreaField('Message', validators=[InputRequired()])
    photo = FileField('Photo')

@app.route('/', methods=['GET', 'POST'])
def index():
    form = MyForm()
    if form.validate_on_submit():
        name = form.name.data
        password = form.password.data
        remember_me = form.remember_me.data
        salary = form.salary.data
        gender = form.gender.data
        country = form.country.data
        message = form.message.data
        photo = form.photo.data.filename
        return f'Name: {name} <br> Password: {generate_password_hash(password)}
<br> Remember me: {remember_me} <br> Salary: {salary} <br> Gender: {gender} <br>
Country: {country} <br> Message: {message} <br> Photo: {photo}'
    return render_template('myform.html', form=form)

if __name__ == '__main__':
    app.run()

```

myform.html

```
<!DOCTYPE html>
<html>
<head>
  <title>My Application</title>
</head>
<body>
  <h1>My Application</h1>
  <form method="post" action="/" enctype="multipart/form-data">
    {{ form.csrf_token }}
    <p>{{ form.name.label }} {{ form.name() }}</p>
    <p>{{ form.password.label }} {{ form.password() }}</p>
    <p>{{ form.remember_me() }} {{ form.remember_me.label }}</p>
    <p>{{ form.salary.label }} {{ form.salary() }}</p>
    <p>{{ form.gender.label }} {{ form.gender() }}</p>
    <p>{{ form.country.label }} {{ form.country() }}</p>
    <p>{{ form.message.label }} {{ form.message() }}</p>
    <p>{{ form.photo.label }} {{ form.photo() }}</p>
    <p><input type="submit" value="Submit"></p>
  </form>
</body>
</html>
```

Explanation:

- Created a Form Class: MyForm inherits from FlaskForm and includes various fields like StringField, PasswordField, BooleanField, etc., with appropriate validators.
- Passed Form to Template: In the index function, an instance of MyForm is created and sent to index.html.
- Handled Form Submission: The validate_on_submit method checks if the form is valid before processing data.
- Retrieved and Displayed Data: If valid, form data is extracted and shown in the browser.
- Secured Passwords: Used generate_password_hash to encrypt passwords for better security.

Output:

My Application

Name

Password

☐ Remember me

Salary

Gender

- ☐ Male
- ☐ Female

Country

Message

Photo No file chosen